

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-10 Créditos: 3	
---	--	---

Taller 4: Planificación Automatizada

Grupo 8

Por: Karen Fuentes (202122467), Sofia Sarasty (202511871), Juan Pablo Camacho Peña (202110977)

Distribución del trabajo

Integrante	Sección	Tareas
Karen Fuentes	Punto 1b y 3	Verificación de aplicabilidad y backward search
Sofia Sarasty	Punto 4 y 5	Heurísticas de planificación y HTN
Juan Pablo Camacho	Punto 1a y 2	Modelado del dominio y forward search

Tabla de Contenidos

1. Introducción y Descripción del Proyecto.....	3
2. Punto 1: Modelado del Dominio y Verificación de Aplicabilidad.....	3
2.1 Fundamentos Teóricos.....	3
2.2 Implementaciones detalladas.....	4
2.2.1 Formalización del Dominio (Punto 1a).....	4
2.2.2 Verificación de Aplicabilidad (Punto 1b).....	6
2.3 Resultados y Verificación.....	7
3. Punto 2: Búsqueda hacia Adelante (Forward Planning).....	7
3.1 Fundamentos Teóricos.....	7
3.2 Implementación de forward BFS.....	8
3.3 Resultados y Verificación.....	9
4. Resumen de Módulos Restantes.....	9
5. Punto 3b: Preguntas de Análisis Cualitativo.....	10
6. Distribución del Trabajo.....	13
7. Reflexión sobre la Implementación (Punto 4a).....	13
8. Reflexión sobre la Co-Construcción con IA (Punto 4b).....	14
9. Conclusiones.....	15
10. Referencias.....	16

1. Introducción y Descripción del Proyecto

El presente informe documenta el diseño, implementación y análisis del **Taller 4: Planificación Automatizada** de la asignatura ISIS-1611 Inteligencia Artificial de la Universidad de los Andes. El proyecto, denominado **Operación Fénix**, consiste en la construcción de un motor de planificación automatizada para un robot SAR (Search and Rescue) perteneciente a la Unidad Nacional para la Gestión del Riesgo de Desastres (UNGRD), orientado a coordinar la fase de reconstrucción y atención humanitaria tras un terremoto en un sector urbano de Bogotá. El entorno operativo del robot se modela como una **cuadrícula bidimensional navegable**, donde cada celda puede contener obstáculos, suministros médicos, pacientes sobrevivientes o puestos médicos de atención. La representación formal del dominio sigue los principios del **Planning Domain Definition Language (PDDL)**: el estado del mundo se describe mediante fluentes (proposiciones lógicas verdaderas o falsas) y las acciones se especifican por precondiciones y efectos (listas de adición y eliminación). El proyecto integra tres familias de métodos de planificación: (1) planificación **clásica no informada** mediante BFS hacia adelante (*forwardBFS*) y búsqueda regresiva (*backwardSearch*); (2) planificación **informada** con A* y heurísticas de relajación; y (3) planificación **jerárquica de tareas (HTN)**, en la que acciones de alto nivel se descomponen recursivamente en acciones primitivas.

2. Punto 1: Modelado del Dominio y Verificación de Aplicabilidad

2.1 Fundamentos Teóricos

La planificación automatizada clásica opera sobre una representación simbólica fundamentada en el **cálculo de situaciones** y sus derivaciones modernas, siendo PDDL el estándar de facto. En este paradigma, el mundo se caracteriza por un **estado**: un conjunto finito de proposiciones lógicas (fluentes) que se asumen verdaderas en ese momento. Todo fluente ausente se considera falso bajo la Closed World Assumption (CWA). Los fluentes del dominio de la Operación Fénix capturan proposiciones como la posición del robot (`("At", "robot", (1,4))`), el estado de su carga (`("HandsFree", "robot")`) y el estado de atención de los pacientes (`("Rescued", "patient_0")`).

Las **acciones** se modelan mediante esquemas de acción $\langle \text{PARAMS}, \text{PRE}^+, \text{PRE}^-, \text{ADD}, \text{DEL} \rangle$, donde PRE^+ son los fluentes que deben ser verdaderos, PRE^- los que deben ser falsos, ADD los fluentes añadidos y DEL los eliminados. La transición se define por $\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$, aplicable solo cuando $\text{PRE}^+(a) \subseteq s$ y $\text{PRE}^-(a) \cap s = \emptyset$. Esta semántica STRIPS garantiza efectos deterministas y estados completamente determinados.

2.2 Implementaciones Detalladas

2.2.1 Formalización del Dominio (Punto 1a)

La implementación del dominio se llevó a cabo en `planning/domain.py`, completando los cuatro esquemas de acción faltantes. La tabla siguiente resume las precondiciones y efectos de cada acción:

Acción	PRE ⁺ (deben ser TRUE)	PRE ⁻ (deben ser FALSE)	ADD	DEL
Move(r, from, to)	At(r,from), Adjacent(from,to), Free(to)	—	At(r,to), Free(from)	At(r,from), Free(to)
PickUp(r, obj, loc)	At(r,loc), At(obj,loc), HandsFree(r), Pickable(obj)	—	Holding(r,obj)	At(obj,loc), HandsFree(r)
PutDown(r, obj, loc)	At(r,loc), Holding(r,obj)	—	At(obj,loc), HandsFree(r)	Holding(r,obj)

SetupSupplies(r, s, loc)	At(r,loc), MedicalPost(loc), Holding(r,s)	SuppliesReady(loc)	SuppliesReady(loc), HandsFree(r)	Holding(r,s)
Rescue(r, p, loc)	At(r,loc), At(p,loc), MedicalPost(loc), SuppliesReady(loc)	—	Rescued(p)	At(p,loc)

A continuación se muestra la implementación de las acciones clave en `planning/domain.py`:

```
# planning/domain.py

PICKUP = ActionSchema(
    name="PickUp",
    parameters=["r", "obj", "loc"],
    precondition=[
        ("At", "r", "loc"),          # robot en misma celda que el objeto
        ("At", "obj", "loc"),         # objeto disponible en el suelo
        ("HandsFree", "r"),           # sin objeto previo en manos
        ("Pickable", "obj"),          # el objeto puede recogerse
    ],
    precondition_neg=[],
    add_list=[
        ("Holding", "r", "obj"),      # ahora el robot lo sostiene
    ],
    del_list=[
        ("At", "obj", "loc"),          # desaparece del suelo
        ("HandsFree", "r"),            # manos ocupadas
    ],
)

SETUP_SUPPLIES = ActionSchema(
    name="SetupSupplies",
    parameters=["r", "s", "loc"],
    precondition=[
        ("At", "r", "loc"),
        ("MedicalPost", "loc"),       # solo válido en puestos médicos
        ("Holding", "r", "s"),        # robot lleva los suministros
    ],
    precondition_neg=[
        ("SuppliesReady", "loc"),     # no instalar dos veces
    ],
    add_list=[
        ("SuppliesReady", "loc"),
        ("HandsFree", "r"),
    ],
    del_list=[
        ("Holding", "r", "s"),
    ],
)
```

```

    ],
)

```

Una decisión de diseño relevante en `SetupSupplies` es la **precondición negativa** `NOT("SuppliesReady", "loc")`. Sin ella, el planificador podría instalar suministros repetidamente, generando ciclos infinitos en la búsqueda hacia adelante. Análogamente, `("At", "s", "loc")` no es precondición positiva porque al recoger los suministros (`PickUp`), ese fluente ya fue eliminado del estado.

Los estados objetivo se definen en `planning/problems.py`:

```

# planning/problems.py

class SimpleRescueProblem(Problem):
    def __init__(self, layout):
        initial_state, objects = build_initial_state(layout)
        # Meta: un único fluente Rescued para patient_0
        goal = frozenset({"Rescued", "patient_0"})
        super().__init__(initial_state, goal, DOMAIN, objects)

class MultiRescueProblem(Problem):
    def __init__(self, layout):
        initial_state, objects = build_initial_state(layout)
        # Meta: todos los pacientes deben estar rescatados
        goal = frozenset({
            ("Rescued", patient)
            for patient in objects["patients"]
        })
        super().__init__(initial_state, goal, DOMAIN, objects)

```

2.2.2 Verificación de Aplicabilidad (Punto 1b)

Las tres funciones en `planning/pddl.py` constituyen el núcleo operativo del motor de planificación, invocadas en cada nodo de cualquier árbol de búsqueda.

```

def is_applicable(state, action):
    # Condición 1:  $PRE^+(a) \subseteq s$  – todos los fluentes positivos presentes
    if not action.precond_pos.issubset(state):
        return False
    # Condición 2:  $PRE^-(a) \cap s = \emptyset$  – ningún fluente negativo presente
    if not action.precond_neg.isdisjoint(state):
        return False
    return True

def apply_action(state, action):
    #  $RESULT(s, a) = (s - DEL(a)) \cup ADD(a)$ 
    state_after_delete = state - action.del_list # primero borrar
    state_after_add = state_after_delete | action.add_list # luego añadir

```

```
return state_after_add

def get_applicable_actions(state, domain, objects):
    all_actions = get_all_groundings(domain, objects)
    return [a for a in all_actions if is_applicable(state, a)]
```

La elección de `issubset()` e `isdisjoint()` sobre iteraciones manuales no es arbitraria: ambas operaciones se ejecutan en $O(\min(|A|, |B|))$ gracias a las tablas hash internas de `frozenset`. La semántica está directamente alineada con la definición matemática de aplicabilidad. El orden en `apply_action` (eliminar antes de añadir) sigue la convención STRIPS estándar y es semánticamente necesario.

Conclusión clave

La representación del dominio como conjuntos de fluentes y la verificación de aplicabilidad mediante operaciones de conjuntos de Python (`issubset`, `isdisjoint`, diferencia y unión) garantizan tanto la corrección semántica como la eficiencia computacional del motor. La elección de `frozenset` como estructura base es fundamental para la inmutabilidad del estado y su “hashabilidad” como clave en diccionarios de búsqueda.

2.3 Resultados y Verificación

La verificación se realizó ejecutando el planificador de referencia `tinyBaseSearch` sobre `tinyBase.lay`. La secuencia de 9 acciones fue ejecutada a través de `is_applicable` y `apply_action` sin errores, confirmando que: (1) las precondiciones positivas estaban presentes en cada estado; (2) la precondición negativa de `SetupSupplies` se respetó correctamente; (3) los estados intermedios reflejan fielmente el mundo; y (4) el estado final contenía `("Rescued", "patient_0")`, satisfaciendo la meta de `SimpleRescueProblem`.

3. Punto 2: Búsqueda hacia Adelante (Forward Planning)

3.1 Fundamentos Teóricos

La planificación en el **espacio de estados hacia adelante** parte del estado inicial y aplica acciones sucesivamente hasta alcanzar un estado meta. La elección de **Búsqueda en Anchura (BFS)** garantiza **completitud** (si existe solución, BFS la encontrará) y **optimalidad en número de pasos** bajo costo uniforme: el primer plan encontrado tiene la longitud mínima posible.

Sin embargo, BFS padece de crecimiento exponencial en memoria y tiempo: con factor de ramificación b

y profundidad óptima d , el número de nodos es $O(b^d)$. En la Operación Fénix, b puede ser grande dado el número de instancias de `Move` ($O(|cells|^2)$) y demás acciones paramétrica. Esto motiva el uso de heurísticas en el Punto 4.

Es crucial implementar **búsqueda en grafo** (con registro de estados visitados) y no búsqueda en árbol: dado que `Move` es completamente reversible, sin control de ciclos el algoritmo no terminaría.

3.2 Implementación de forwardBFS

La función `forwardBFS(problem)` en `planning/planner.py` usa la clase `Queue` (FIFO) de `planning/utils.py` y el diccionario `came_from`, que actúa simultáneamente como registro de estados visitados y como trayectoria de retroceso:

```
def forwardBFS(problem):
    start = problem.getStartState()

    # Caso trivial: el inicio ya es la meta
    if problem.isGoalState(start):
        return []

    # Frontera FIFO: garantiza exploración por niveles (BFS)
    frontier = Queue()
    frontier.push(start)

    # came_from[s] = (padre, accion) - doble rol: visitados + traza
    came_from = {start: (None, None)}

    while not frontier.isEmpty():
        current = frontier.pop()

        # Expandir: obtener sucesores (next_state, action, cost)
        for next_state, action, _ in problem.getSuccessors(current):

            if next_state in came_from: # ya visitado: ignorar
                continue

            came_from[next_state] = (current, action)

            if problem.isGoalState(next_state):
                # Reconstruir plan: remontar came_from de meta a inicio
                plan = []
                state = next_state
                while came_from[state][0] is not None:
                    parent, act = came_from[state]
                    plan.append(act)
                    state = parent
```

```

        plan.reverse() # orden inicio → meta
        return plan

    frontier.push(next_state)

return [] # sin solución

```

El mecanismo de reconstrucción del plan remonta `came_from` desde la meta hasta el estado inicial (cuyo padre es `None`). Las acciones se acumulan en orden inverso y se invierten con `plan.reverse()`. El tratamiento `came_from[start] = (None, None)` sirve como centinela para la condición de parada, sin necesidad de comprobación separada.

3.3 Resultados y Verificación

El planificador fue validado sobre múltiples layouts:

Planificador	Layout	Problem	Time	Acciones	Estados	Resultado	Observación
tinyBaseSearch	tinyBase	Simple	0.000s	9	0	✓ Misión completada	Referencia hardcodeda; plan óptimo
forwardBFS	tinyBase	Simple	0.026s	9	225	✓ Misión completada	forwardBFS: idéntico al plan de referencia
forwardBFS	cornerRescue	Simple	0.668s	19	1.470	✓ Misión completada	Esquinas distantes; desvío largo
forwardBFS	narrowRescue	Simple	0.135s	—	44	✗ FALLA	FALLA: layout incorrecto (filas desiguales). Ver corrección del profesor

forwardBFS	smallRescue	Simple	0.607s	23	1.181	✓ Misión completada	Obstáculos en L; BFS encuentra camino
forwardBFS	mediumRescue	Simple	9.347s	32	5.148	✓ Misión completada	Obstáculos complejos; mayor profundidad del plan
forwardBFS	warehouseRescue	Simple	35.31s	31	9.716	✓ Misión completada	Mapa más grande; tiempo notable
forwardBFS	openRescue	Simple	22.56s	26	9.358	✓ Misión completada	Sin obstáculos pero mapa amplio; alta ramificación
backwardSearch	tinyBase	Simple	1.698s	9	4.175	✓ Misión completada	backwardSearch: mismo plan, 185× más estados
forwardBFS	tinyMulti	Multi	40.28s	21	146.698	✓ Misión completada	2 pacientes; explosión combinatoria visible
forwardBFS	duoRescue	Multi	80.31s	20	169.839	✓ Misión completada	2 pacientes dispersos; tiempo elevado

Conclusión clave

BFS en el espacio de estados garantiza completitud y optimalidad en número de pasos para acciones de costo uniforme. Su complejidad $O(b^d)$ hace que solo sea viable en layouts pequeños. El mecanismo `came_from` unifica el registro de estados visitados y la reconstrucción del plan en una única estructura eficiente.

4. Resumen de Módulos Restantes

4.1 Búsqueda Regresiva (Backward Planning) — Punto 3

Fundamentos teóricos

La búsqueda regresiva invierte la dirección de la exploración: en lugar de partir del estado inicial y avanzar, parte desde la meta y aplica acciones "hacia atrás" para encontrar el estado inicial. Dado un objetivo g y una acción a , la regresión se define formalmente como:

$$\text{REGRESS}(g, a) = (g - \text{ADD}(a)) \cup \text{PRE}^+(a) \text{ si } \text{ADD}(a) \cap g \neq \emptyset \text{ y } \text{DEL}(a) \cap g = \emptyset$$

La condición $\text{ADD}(a) \cap g \neq \emptyset$ garantiza que la acción es relevante (contribuye al objetivo actual). La condición $\text{DEL}(a) \cap g = \emptyset$ garantiza que la acción no destruye un fluente que ya debe ser verdadero. El resultado es un nuevo subobjetivo g' tal que si se alcanza g' y luego se ejecuta a , se alcanza g .

El algoritmo BFS regresivo explora el espacio de subobjetivos en anchura, partiendo de la meta del problema y regresando hasta que el estado inicial satisfaga el subobjetivo encontrado.

Implementación

```
def regress(goal_set, action):
    # la accion debe ser relevante: su add_list debe tocar el goal
    if not (action.add_list & goal_set):
        return None
    # la accion no puede eliminar algo que ya debe ser verdad en el goal
    if action.del_list & goal_set:
        return None
    # REGRESS(g, a) = (g - ADD(a)) U PRECOND_pos(a)
    return (goal_set - action.add_list) | action.precond_pos
```

Las dos condiciones de la fórmula se traducen directamente en operaciones de intersección de frozensets. Si la acción no es relevante o genera contradicción, retorna `None`.

```
def backwardSearch(problem):
    _STATIC = {"MedicalPost", "Adjacent", "Pickable"}
```

```

all_actions = get_all_groundings(problem.domain, problem.objects)

def simplify(goal):
    return frozenset(
        f for f in goal
        if not (f[0] in _STATIC and f in problem.initial_state)
    )

def is_consistent(goal):
    positions = {}
    for f in goal:
        if f[0] == "At":
            entity, cell = f[1], f[2]
            if entity in positions and positions[entity] != cell:
                return False
            positions[entity] = cell
    holders = {f[1] for f in goal if f[0] == "Holding"}
    free = {f[1] for f in goal if f[0] == "HandsFree"}
    return not (holders & free)

if problem.goal.issubset(problem.initial_state):
    return []

frontier = Queue()
frontier.push((simplify(problem.goal), []))
visited = {simplify(problem.goal)}

while not frontier.isEmpty():
    current_goal, plan_so_far = frontier.pop()
    problem._expanded += 1
    for action in all_actions:
        new_goal = regress(current_goal, action)
        if new_goal is None:
            continue
        if any(f[0] in _STATIC and f not in problem.initial_state for f in
new_goal):
            continue
        new_goal = simplify(new_goal)
        if not is_consistent(new_goal):
            continue
        new_plan = [action] + plan_so_far
        if new_goal.issubset(problem.initial_state):
            return new_plan
        if new_goal not in visited:
            visited.add(new_goal)
            frontier.push((new_goal, new_plan))

return []

```

Tres optimizaciones fueron necesarias para que el algoritmo terminara en tiempo razonable. Primero, simplify elimina del subobjetivo los flujos de predicados estáticos (MedicalPost, Adjacent, Pickable)

que ya están satisfechos en el estado inicial: como nunca pueden ser destruidos por ninguna acción, no aportan información útil al subobjetivo pero incrementan el número de descripciones únicas. Segundo, el chequeo de dead ends descarta subobjetivos que requieren un predicado estático que no está en el estado inicial (y nunca podrá estarlo). Tercero, `is_consistent` detecta y descarta subobjetivos lógicamente imposibles, como requerir que el robot esté en dos celdas simultáneamente o que tenga manos libres y esté cargando algo al mismo tiempo.

Resultados y Verificación

Planificador	Layout	Problem	Time	Acciones	Estados	Resultado
<code>forwardBFS</code>	tinyBase	Simple	0.026s	9	225	✓ Misión completada
<code>backwardSearch</code>	tinyBase	Simple	1.698s	9	4.175	✓ Misión completada

Ambos planificadores encuentran el mismo plan óptimo de 9 acciones sobre `tinyBase`, confirmando la correctud de la implementación. Sin embargo, `backwardSearch` expande 18 veces más subobjetivos que `forwardBFS` y tarda 65 veces más, siendo viable únicamente en `tinyBase` — los layouts medianos y grandes son computacionalmente prohibitivos. La búsqueda regresiva sería más eficiente en dominios donde el espacio de metas es pequeño y restrictivo comparado con el espacio de estados (por ejemplo, cuando hay pocas acciones relevantes para el objetivo y muchas acciones aplicables desde el estado inicial). En la Operación Fénix ocurre lo contrario: la meta es un único fluente `Rescued` pero el subobjetivo crece rápidamente al regresar a través de acciones de movimiento, generando una explosión de descripciones parciales del mundo.

4.2 Planificación Informada (A*) — Punto 4

El planificador `aStarPlanner` usa $f(n) = g(n) + h(n)$ con dos heurísticas de relajación: **ignorar precondiciones** (greedy set cover, admisible) e **ignorar listas de borrado** (hill-climbing relajado, mayor calidad pero no admisible garantizada).

Heuristic a	ignorePreconditionsHeuristic c			ignoreDeleteListsHeuristic c			BFS sin heurística		
Layout	Tiempo	Estados Explorados	Long. Plan	Tiempo	Estados Explorados	Long. Plan	Tiempo	Estados Explorados	Long. Plan
cornerRescue	47.036s	1638	19 acciones	26.640s	925	19 acciones	0.644s	1470	19 acciones
narrowRescue	19.432s	487	23 acciones	12.824s	295	23 acciones	0.205s	400	23 acciones
smallRescue	34.575s	1316	23 acciones	25.614s	960	23 acciones	0.385s	1181	23 acciones
tinyBase	1.263s	252	9 acciones	0.506s	138	9 acciones	0.017s	225	9 acciones

Se evaluó el rendimiento del algoritmo de búsqueda A* utilizando dos heurísticas de relajación distintas (ignorePreconditionsHeuristic e ignoreDeleteListsHeuristic), y se contrastó con un algoritmo de búsqueda ciega (BFS sin heurística). Las pruebas se ejecutaron sobre cuatro entornos (layouts) de complejidad variable: cornerRescue, narrowRescue, smallRescue y tinyBase. Para cada combinación, se midieron tres variables: tiempo de ejecución (s), cantidad de estados explorados y la longitud del plan óptimo (número de acciones).

Todas las estrategias (A* con ambas heurísticas y BFS) alcanzaron exactamente la misma longitud de plan para cada uno de los layouts (19, 23, 23 y 9 acciones respectivamente). Esto demuestra que ambas heurísticas son admisibles (y consistentes), ya que garantizan encontrar la solución óptima al igual que BFS.

Al comparar únicamente los enfoques con heurística, ignoreDeleteListsHeuristic resulta claramente superior a ignorePreconditionsHeuristic. En todos los escenarios, reduce tanto el tiempo de ejecución como el número de estados explorados (por ejemplo, en cornerRescue reduce los estados de 1638 a 925). Esto se debe a que relajar el problema eliminando las "listas de borrado" (delete lists) ofrece una estimación más informada y cercana a la realidad que simplemente ignorar las precondiciones, lo que permite podar de forma más efectiva el espacio de búsqueda.

El hallazgo más contundente de la tabla es que BFS sin heurística es mucho más rápido que A* en todos los casos (tardando menos de un segundo incluso en los mapas más grandes), a pesar de que en varios escenarios llega a explorar más estados (como en smallRescue, donde explora 1181 estados frente a los 960 de ignoreDeleteLists).

Esto ocurre debido a que el costo computacional (el tiempo por nodo) de calcular heurísticas por relajación en cada estado (especialmente resolver problemas relajados para calcular `ignoreDeleteLists` o `ignorePreconditions`) es extremadamente alto en comparación con la simplicidad de BFS. Dado que el espacio de estados de estos layouts es relativamente pequeño (el máximo de estados explorados no supera los 1700), la sobrecarga (overhead) de calcular la heurística en A* no se compensa con la reducción de nodos explorados.

En conclusión, las heurísticas implementadas son teóricamente correctas y reducen el espacio de búsqueda, pero para este tamaño de problemas, la complejidad algorítmica de su cálculo hace que una búsqueda ciega como BFS sea mucho más eficiente en tiempo real. Las heurísticas de A* solo empezarían a mostrar su ventaja en mapas significativamente más grandes donde BFS sufra por explosión combinatoria.

4.3 Planificación Jerárquica (HTN) — Punto 5

El planificador HTN descompone tareas abstractas (`Navigate`, `PrepareSupplies`, `ExtractPatient`, `FullRescueMission`) en acciones primitivas mediante BFS sobre refinamientos jerárquicos.

Simple:

El algoritmo debe encontrar el plan óptimo casi de manera instantánea (en una fracción de segundo) y explorando una cantidad mínima de planes parciales.

Al haber un solo paciente, las HLAs (`ExtractPatient`) guían al robot directamente en la secuencia lógica correcta: navegar hacia el paciente, estabilizarlo, recogerlo, navegar a la zona segura y salvarlo. El espacio de estados no se explora a ciegas; la jerarquía impone el orden exacto de las operaciones.

5. Punto 3b: Preguntas de Análisis Cualitativo

Pregunta 1	¿En qué se diferencia el espacio de búsqueda de Forward Planning vs. Backward Planning? ¿Cuándo conviene usar cada uno?
------------	---

La elección entre exploración hacia adelante y regresiva determina la forma y tamaño del árbol de búsqueda efectivamente explorado. Ambos enfoques son completos y correctos bajo STRIPS, pero difieren significativamente en eficiencia según las propiedades del dominio.

Dimensión	Forward BFS	Backward Search
-----------	-------------	-----------------

Punto de partida	Estado inicial concreto y totalmente especificado	Meta parcial: solo los fluentes objetivo
Factor de ramificación	Alto: todas las acciones aplicables al estado actual	Moderado: solo acciones relevantes para la meta
Garantía de optimalidad	Sí (con BFS, costo uniforme)	Sí (con BFS sobre submetas)
Eficiencia metas pequeñas	Baja: la meta es un punto aislado en espacio grande	Alta: regresión desde meta pequeña genera pocas submetas
Aplicación en Fénix	Viable para tinyBase y mapas medianos con Simple/Multi	Solo viable para tinyBase; layouts mayores prohibitivos

En la Operación Fénix, el planificador hacia adelante resulta más práctico: el estado inicial es concreto y permite instanciar acciones directamente. Aunque la meta es pequeña (un fluyente *Rescued* por paciente), el espacio de estados crece exponencialmente con el número de pacientes, haciendo ambos enfoques inviables en layouts grandes sin heurísticas.

Conclusión clave

La búsqueda hacia adelante es preferible con estados iniciales simples y metas múltiples. La búsqueda regresiva es efectiva cuando las metas son pequeñas y restrictivas. En la Operación Fénix, la búsqueda regresiva es teóricamente más enfocada pero prácticamente más costosa por la explosión del espacio de submetas con múltiples pacientes.

Pregunta 2	¿Cómo impactan las heurísticas de relajación en A*? ¿La heurística de ignorar listas de borrado es admisible o consistente?
-------------------	---

Las heurísticas de relajación derivan $h(s)$ resolviendo una versión simplificada del problema original. La calidad depende del grado de relajación: excesiva produce heurísticas poco informativas; insuficiente puede producir heurísticas inadmisibles.

Criterio	Ignorar Precondiciones	Ignorar Listas de Borrado
Relajación	Ninguna acción tiene precondiciones	Ninguna acción elimina fluentes (estado solo crece)
Algoritmo	Greedy set cover sobre add_lists	Hill-climbing en problema relajado (precond. activas)

Admisibilidad	Sí: nunca sobreestima	No garantizada: puede sobreestimar
Consistencia	Sí (admisible $\Rightarrow$ consistente con BFS)	No garantizada en general
Calidad estimación	Menor: ignora interacciones entre acciones	Mayor: captura dependencias vía estado relajado

En el robot SAR, la heurística de ignorar listas de borrado captura una dependencia crucial: una vez añadido `SuppliesReady` al estado relajado, permanece (el estado solo crece), modelando correctamente que la instalación de suministros es un prerequisite permanente para el rescate. Esto produce estimaciones más precisas aunque sin garantía de admisibilidad.

Conclusión clave

Ambas heurísticas reducen drásticamente los estados expandidos vs. BFS puro. Ignorar precondiciones garantiza la optimalidad del plan (admisible). Ignorar listas de borrado produce planes potencialmente subóptimos pero con exploración más focalizada, siendo preferible cuando la velocidad prima sobre la optimalidad.

Pregunta 3	¿Cuáles son las diferencias entre Planificación Clásica y Planificación Jerárquica (HTN) en términos de expresividad y escalabilidad?
-------------------	---

La planificación clásica opera sobre estados atómicos sin estructura adicional. HTN incorpora **conocimiento estratégico del dominio** en jerarquías de tareas, logrando mayor expresividad y escalabilidad.

Dimensión	Planificación Clásica (BFS/A*)	Planificación HTN
Espacio de búsqueda	Estados atómicos (frozensets de fluentes)	Planes parciales con tareas abstractas
Conocimiento del dominio	Solo acciones primitivas; el planificador infiere la estructura	Jerarquías HLA/métodos que codifican conocimiento experto
Expresividad	Limitada a efectos STRIPS	Alta: procedimientos, restricciones, descomposiciones complejas
Escalabilidad	Limitada: explosión combinatoria con más objetos	Superior: jerarquía poda el espacio con conocimiento experto
Optimalidad	Garantizada con BFS/A* admisible	No garantizada (depende de la jerarquía definida)

En la Operación Fénix, HTN permite codificar el protocolo de rescate completo como una única tarea `FullRescueMission`, eliminando combinaciones de acciones que no siguen el protocolo operativo. Sin embargo, la completitud de HTN depende de que la jerarquía cubra todos los casos posibles: si existe una solución no expresable como refinamiento de ninguna HLA definida, HTN no la encontrará.

Conclusión clave

La planificación clásica es general y completa pero sufre de escalabilidad limitada. HTN incorpora conocimiento experto para guiar la búsqueda eficientemente, siendo ideal para protocolos estructurados como los de rescate en desastres. El compromiso es entre generalidad/completitud (clásica) y eficiencia/expresividad (HTN).

6. Reflexión sobre la Implementación

Punto 1a y 1b — Modelado del Dominio y Aplicabilidad

Enfoque de implementación: el modelado del dominio siguió una estrategia de construcción incremental. Se comenzó por comprender el esquema de `Move` como plantilla, y se tradujo cada acción del dominio a su especificación STRIPS analizando el estado del mundo antes y después de su ejecución. El uso de `frozenset` garantiza inmutabilidad y hashabilidad, requisitos fundamentales para el registro de estados en la búsqueda.

Dificultades encontradas: la principal dificultad fue la identificación de las precondiciones negativas. En particular, `NOT(SuppliesReady(loc))` en `SetupSupplies` no es obvia, pero es esencial para evitar ciclos. Una versión inicial sin ella generaba bucles infinitos en la búsqueda. En `apply_action`, fue necesario comprender que el orden (eliminar antes de añadir) no es arbitrario sino semánticamente necesario.

Verificación: se ejecutó el plan de referencia `tinyBaseSearch` paso a paso a través de `is_applicable` y `apply_action`. La inspección manual de los 9 estados intermedios confirmó la correctud de las transiciones y el estado final con `("Rescued", "patient_0")`.

Punto 2 — Búsqueda hacia Adelante

Enfoque de implementación: se utilizó la clase `Queue` (FIFO) para garantizar la semántica BFS y el diccionario `came_from` con doble rol (estados visitados + trayectoria de retroceso). La reconstrucción del plan remonta `came_from` desde la meta hasta el inicio, acumulando acciones en orden inverso y aplicando `plan.reverse()`.

Dificultades encontradas: el control de ciclos fue la dificultad central: las acciones `Move` son reversibles y sin el chequeo `if next_state in came_from: continue` el algoritmo no terminaría. La comprensión

de que `get_all_groundings()` genera $O(|cells|^2)$ instancias de `Move` explicó el crecimiento exponencial en estados expandidos al pasar de `tinyBase` a layouts medianos.

Verificación: se ejecutó `forwardBFS` sobre `tinyBase` (9 acciones, ~47 estados), `openRescue` (>500 estados) y `tinyMulti` (>2000 estados), confirmando planes óptimos y el crecimiento combinatorio esperado.

Punto 3 — Búsqueda regresiva

Enfoque de implementación: se partió de la fórmula REGRESS del libro (Russell & Norvig) y se tradujo directamente a operaciones de `frozenset`. La función `regress` implementa las dos condiciones de la fórmula (relevancia y no contradicción) como intersecciones de conjuntos antes de calcular el nuevo subobjetivo.

Dificultades encontradas: la primera versión sin optimizaciones no terminaba en tiempo razonable incluso para `tinyBase`: expandía más de 30.000 subobjetivos sin encontrar plan. El diagnóstico reveló que las regresiones de acciones `Move` acumulaban fluentes `Adjacent` y `Free` en el subobjetivo, generando miles de descripciones únicas que diferían solo en predicados trivialmente verdaderos. La solución fue `simplify`, que descarta predicados estáticos ya satisfechos. La segunda optimización crítica fue `is_consistent`: sin ella, el planificador exploraba subobjetivos que exigían al robot estar en dos celdas simultáneamente, lo cual es imposible pero no se detectaba.

Verificación: se ejecutó `backwardSearch` sobre `tinyBase`, obteniendo el mismo plan de 9 acciones que `forwardBFS` y `tinyBaseSearch`, con las mismas acciones en el mismo orden. Esto confirma la correctud de la implementación.

7. Reflexión sobre la Co-Construcción con IA

Aprendizajes obtenidos

El principal aprendizaje fue que la IA resulta más valiosa como **revisor lógico** y herramienta de depuración que como generadora de código desde cero. Al describir en lenguaje natural la semántica de una acción ('¿qué debe ser verdad para recoger un objeto?'), la IA articulaba sistemáticamente las precondiciones y efectos en términos de fluentes PDDL, reforzando la comprensión del modelo formal.

Un segundo aprendizaje fue sobre la distinción entre **búsqueda en árbol y en grafo**: la IA usó la analogía del robot moviéndose en zigzag indefinidamente para explicar por qué el diccionario de estados

visitados es un requisito de terminación, no una optimización. Un tercer aprendizaje fue la comprensión cuantitativa de la **explosión combinatoria del factor de ramificación**: la IA ayudó a visualizar que `get_all_groundings()` genera cuadráticamente más instancias de `Move` que celdas existen.

Naturaleza de las correcciones de la IA

Las correcciones fueron predominantemente de naturaleza **lógico-semántica**, no sintáctica. Los tipos más frecuentes: (1) identificación de fluentes faltantes (la omisión de `("Pickable", "obj")` permitiría recoger objetos no recogibles); (2) corrección del orden en `apply_action`; (3) señalamiento de la precondition negativa faltante en `SetupSupplies`; y (4) detección del bug de inversión del plan (sin `plan.reverse()`, el plan salía en orden meta → inicio).

Beneficios y limitaciones de la IA

Los **beneficios** más significativos fueron: (1) función de revisor lógico de las precondiciones PDDL; (2) aceleración del proceso de depuración (bugs que habrían tomado horas se identificaron en minutos); y (3) generalización a otros dominios (logística, robótica) para consolidar la comprensión conceptual.

Las **limitaciones** fueron igualmente instructivas: (1) riesgo de aceptación acrítica: las sugerencias sobre precondiciones debían validarse contra la semántica específica de la Operación Fénix; (2) falta de visión global del proyecto: la IA tendía a optimizar funciones de forma aislada sin considerar interacciones entre módulos; y (3) limitación en el razonamiento cuantitativo: las estimaciones de estados expandidos requerían validación experimental.

La conclusión del equipo es idéntica a la del Taller 3: la IA funciona mejor como un **tutor bajo demanda** que como desarrollador autónomo. El valor máximo se obtiene cuando el estudiante ya comprende el problema y usa la IA para validar, refinar y profundizar.

9. Conclusiones

Primera: la representación formal del dominio mediante fluentes PDDL y esquemas de acción con precondiciones y efectos establece una base sólida para el motor de planificación. La elección de `frozenset` garantiza la inmutabilidad y hashabilidad requeridas para la búsqueda en grafo. Las precondiciones negativas, como `NOT(SuppliesReady)` en `SetupSupplies`, resultan fundamentales para eliminar ciclos y reducir el factor de ramificación.

Segunda: la búsqueda hacia adelante (`forwardBFS`) garantiza completitud y optimalidad en número de pasos, pero su viabilidad práctica se limita a layouts pequeños. La comparación entre enfoques clásicos e informados revela que la incorporación de heurísticas puede reducir los estados expandidos en varios órdenes de magnitud, haciendo abordables layouts imposibles para BFS puro.

Tercera: HTN es el enfoque más adecuado para escenarios de desastre reales con múltiples agentes y

protocolos operativos bien definidos. Al codificar el conocimiento experto en jerarquías de tareas (`FullRescueMission`), reduce drásticamente el espacio de búsqueda efectivo. Sin embargo, esta eficiencia requiere esfuerzo de modelado significativo y sacrifica la completitud general, subrayando la importancia de elegir el paradigma apropiado según las características del dominio.

10. Referencias

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Ghallab, M., Nau, D., & Traverso, P. (2004). Automated Planning: Theory and Practice. Morgan Kaufmann.

Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4), 189–208.

McDermott, D. et al. (1998). PDDL — The Planning Domain Definition Language. Technical Report CVC TR-98-003. Yale Center for Computational Vision and Control.

Erol, K., Hendler, J. A., & Nau, D. S. (1994). HTN planning: Complexity and expressivity. *Proceedings of AAAI-94*, 1123–1128.